

OPEN-SOURCE · PHARMAVERSE · SAS

Building an Open-Source Pharmaceutical SAS Forum that Mirrors Pharmaverse

Paul F. Robertson, JD, MPH

8 September 2026 · Proof-of-concept: safety ADaM SAS analogues of Admiral

<https://github.com/probert7-git2/pharmaverse-admiral-sas-macros>

The vision

An open-source SAS macro library for clinical statistical programming

Mirror pharmaverse

- Same derivation *ideas* as R {`admiral`} / pharmaverse
- Modular, validated SAS macros (%m_*) that parallel Admiral functions
- AI-assisted development (Cursor LLM) to accelerate reverse-engineering

Core purpose

- **Independent oversight** — emerging biotechs QA CRO deliverables (Sponsors retain regulatory responsibility)
- **Dual-language auditing** — reliable cross-check between R and SAS: a form of “double programming” that goes beyond two programmers using the *same* language

Unlike same-language double programming, this cross-check treats **pharmaverse / Admiral as the repository of truth**; SAS macros are proven against that R gold standard.

Industry context

Navigating shifts across pharma & statistical computing

- **Open-source shift to R** — Roche, GSK, J&J, AstraZeneca and others have embraced pharmaverse in recent years
- **Evolving landscape** — R is central to Pharmaverse today; possibly Python may replace it in the future
- **SAS remains rooted** — deeply reliable for data management and marketing-application submissions

Implication: Sponsors need a path that keeps modern open-source design *and* long-lived SAS continuity—not an either/or choice.

The Gap — why SAS still matters

July 31 framing — open-source instability, R costs, who stays on SAS

Unmatched code stability

- The main open-source issue is **instability**: R functions are sometimes deprecated, so a script that runs today may fail in 12 months
- SAS keeps exceptional backward compatibility (e.g., PROC SQL era → still runs)
- Health Authority questions years later: language churn is a cost SAS often avoids

True total cost of ownership

- R is free—but GxP qualification & upkeep are not
- Ongoing labor + infrastructure for a validated R estate
- **Smaller companies stay on SAS** — in-house programming, and for QA of CRO-generated work (the Sponsor still owns the submission)

Proof-of-concept journey

Phase 1 — Exploration

- AI-assisted R→SAS reverse-engineering of functions from **admiralongo** and **xportr**
- Hypothetical bladder-cancer efficacy path
- ADSL, ADRS, ADTTE as first test bed

Phase 2 — Current focus

- Safety-monitoring ADaM: ADSL, ADAE, ADLB, ADVS, ADCM, ADEG
- **1:1 mirroring of Admiral functions** needed for safety ADaM (treatment dates, TRTEMFL, baseline/change, toxicity grades, ...)
- What ran: **edge-case checks, golden-subject testing, improved traceability** (as-programmed CSV → Excel pva_as_programmed_spex.xlsx)
- As-programmed vs pre-specified Admiral programming spec was **planned** but **could not be executed** (no spec found on either side). Work was **curtailed**.

Both the PVA gold standard and the reverse-engineered SAS lack pre-specified spex. Those Admiral specs must have existed internally for PVA/template authors — they were *not available* to this exploration. In industry, pre-specified spex *would* be contrasted with as-programmed / as-executed spex.

Target safety datasets

Core ADaM domain scope (Phase 2)

- **ADSL** — subject-level foundation
- **ADTTE** — time-to-event (AE milestones)
- **ADVS** — vital signs
- **ADEG** — ECG / electrocardiogram
- **ADAE** — adverse events
- **ADLB** — laboratory findings
- **ADCM** — concomitant medications

Technical note: Builders call reusable 1:1 Admiral-function ports (%m_derive_vars_dtm, %m_derive_var_trtemfl, %m_derive_var_base, %m_derive_var_atoxgr*, etc.).

Engineering & macro build process

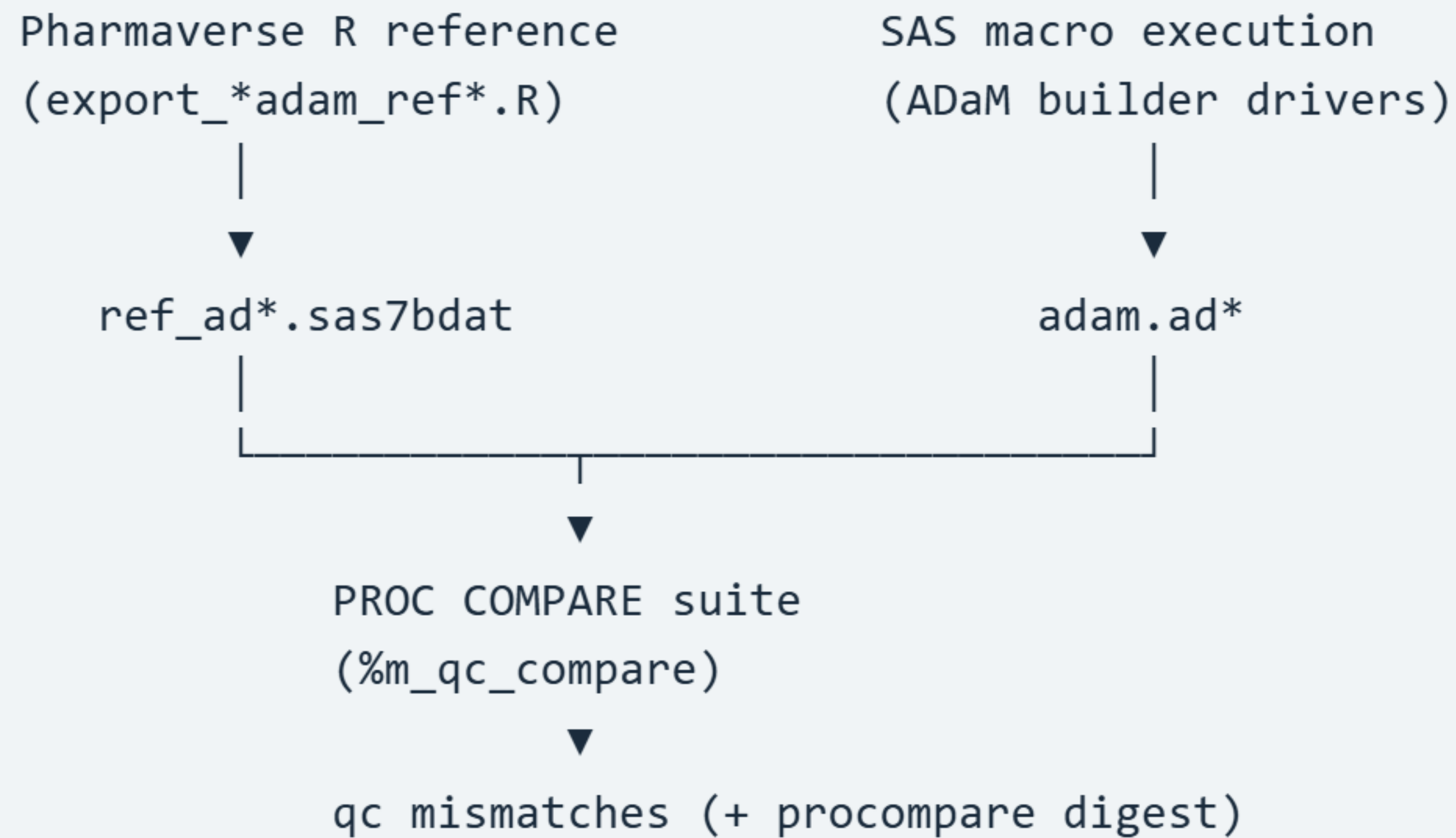
AI-assisted development workflow

1. **Logic ingestion** — analyze R {admiral} source functions, templates, and CDISC specifications
2. **Code translation** — Cursor LLM drafts modular SAS macros (%m_*)
3. **Execution pipelines** — master drivers on SAS OnDemand for Academics (academic runtime, not the project identity)
4. **Log scan** — %m_chklog scans SAS logs for ERROR/WARNING and related issues (one layer only)

Full QC is broader than log checking (next slides): (1) SAS ADaM vs pharmaverse R ADaM from the same SDTM; (2) golden-subject / messy-data tests; (3) as-programmed / as-executed specs — a compare to a pre-specified Admiral programming spec was **planned but curtailed** (no spec found).

Admiral function docs + templates → Cursor drafts %m_* → SAS workflow → multi-layer QC suite

QC Part 1 — Compare against R standards



- **Same inputs** — both paths start from the same pharmaverse SDTM (and shared metadata where applicable)
- **R as gold standard** — pharmaverse / Admiral outputs are the repository of truth
- **Automated auditing** — values, types, lengths, keys, sort order

QC Part 2 — Golden subjects & edge cases

Objective

Catch errors that arise from *data* issues—not only clean pilot rows.

Scenarios tested

- Incomplete, inconsistent, or missing dates
- Missing data; values outside controlled terminology
- Overlapping meds, concurrent AEs, unscheduled visits

Regression guard

Golden-subject suite re-runs after macro updates to catch logic regressions.

Also in the suite

- Edge tests — keys, duplicates, flags, nobs
- Missingness — Required / Expected vars
- Log scan via %m_chklog (reporting layer)

Together with Part 1, this is dual-language QC that goes well beyond traditional same-language double programming.

QC Part 3 — Automated metadata & traceability

As-programmed / as-executed specs — the pre-spec compare was planned, then curtailed

Metadata	Function & content
Variable inventory	Target vars, labels, lengths, types
Derivation registry	Which SAS macro derives each variable (logic as built)
Source-to-target trace	SDTM fields → ADaM targets

Most distinctive check (planned, not executed): compare those machine-generated as-programmed specs to a pre-specified Admiral programming specification. **Could not be executed** — no pre-specified Admiral programming spec was found (for the PVA gold standard or for the SAS side). Work was **curtailed**. In industry, that contrast *would* be the check.

Technical pitfalls we engineered around

- **Date epochs** — R Date = days since 1970; SAS = since 1960 (~3653-day silent shift). Restore Date before XPT; compare calendars carefully.
- **SAS vs R date-scale DTM** — QC pairing resolved (normalize working copies). PVA still *stores* date-scale DTM; do not rewrite SAS datetime to chase the file.
- **ODA work-arounds** (where pertinent) — no DATALINES inside macros; no nested open-code %IF; * comments end at first ;. ODA is student/academic, not production.
- **HGB unit extract** — scan of the trailing) in PARAM (e.g. Hemoglobin (mmol/L)).
- **CTCv4 vs v5** — PVA/template is v4. Hypokalemia G1=G2 → worse-case G2. Hyponatremia: no G2.
- **ADEG two EG files** — leftover EGSTRESU / EGTPT. Source mismatch, not a coding defect.
- **BRTHDTC** — dropped from many DM extracts for privacy; missing on Track A DM. SAS omits BIRTHDT when DM has none.
- **SCREEN AVISITN blank** — non-unique business key is *not* a FAIL (template blanks SCREEN / UNSCHED / ...; tiebreak on SEQ).
- **ADCM TRTEMFL** — OCCDS (after PVA). PVA uses ONTRTFL / PREFL / FUPFL. AVALU dropped — unit lives in PARAM.

Part B — Exploration framing

After July 31: reverse-engineer Admiral for safety ADaM vs published PVA

What this is

- AI-assisted reverse-engineering of Pharmaverse **Admiral** functions for safety-related ADaM
- An **exploration**, not a production library hand-off
- QC against published pharmaverseadam (PVA) — a gold standard industry usually does not have

What this is not

- Not “an ODA project.” ODA is the student/academic runtime some SAS jobs used.
- Teal safety review was prior learning / out of scope — not a deliverable here.
- Not a claim that a pre-specified Admiral programming spec was in hand.

1:1 SAS-macro analogues of Admiral functions

- Reusable SAS macros that **mirror the same Admiral call** — `derive_vars_dtm`, `derive_var_trtemfl`, `derive_var_base`, `derive_var_atoxgr`, `derive_param_qtc`, ...
- Builders: `%m*_pva` / `%m*_admiral_mirror` for ADSL, ADAE, ADVS, ADLB, ADCM, ADEG
- Early lesson: a more global, Tidyverse-heavy port is **not** 1:1. Independent SAS can quietly implement a *different rule*.
- Phase 2 needed that 1:1 mirroring for every Admiral function used in safety ADaM — otherwise QC vs PVA cannot tell “same derivation” from “lucky match.”

QC digest — PVA gold standard vs SAS analogues

Dataset	n (both)	Orphans	PROC COMPARE
ADSL	306	0	PASS
ADAE	1191	0	POLICY
ADVS	65032	0	PASS
ADLB	83652	0	PASS
ADCM	7510	0	POLICY
ADEG	78756	0	FAIL

ADAE · POLICY

Near-match, not PASS. Last-dose 81 mg omitted in PVA; *DTM compared at date precision. Next slide.

ADCM · POLICY

One AENDT row: PVA year-end without EOS cap vs Admiral
max_dates=DTHDT, EOSDT. Keep SAS.

ADEG · FAIL

Two EG files (EGSTRESU / EGTPT). Source, not coding. Left as FAIL on purpose.

ADAE — POLICY, not PASS

Near-match on keys and analysis values vs PVA

ADAE is a near-match; **POLICY**, not PASS, because:

1. PVA last-dose history omits 81 mg titration (a possible teaching-data error?); and
2. *DTM variables are compared at only Date precision — because PVA has date, not datetime values.

Numeric PROC COMPARE can still be 0 diffs if SAS copies the PVA last-dose shortcut and QC compares *DTM as dates. That is not a clinical PASS.

Layer 3 — improved traceability

What we could build without a pre-specified Admiral programming spec

Layer 1 — keys

Merge SAS vs PVA on analysis keys.
Orphans, row counts, chkvars.

Layer 2 — PROC COMPARE

Value digest: PASS / POLICY / FAIL.
Date-scale *DTM normalized on
working copies only.

Layer 3 — trace

As-programmed notes vs Admiral
template steps.
Not a substitute for a missing
programming spec.

Admiral ships function documentation + templates — **not** a CDISC-style ADaM spec spreadsheet. PVA is generated from those templates. Layer 3 is the machine-readable as-programmed side.

As-programmed CSV → Excel worksheet

- First cut: SAS/docs/pva_as_programmed_spex.csv — what SAS actually calls
- Converted to an Excel worksheet: SAS/docs/pva_as_programmed_spex.xlsx
- Joins to as-built CONTENTS after Create ADaM (BOTH / CODE_ONLY / REGISTRY_ONLY)
- prespec_analogue here means the Admiral *template step name* — not a pre-specified programming spec we had in hand
- A spreadsheet compare of as-programmed vs a pre-specified Admiral programming spec was **planned** but **could not be executed** (no spec found). Curtailed. In industry that contrast would be the next check.

Silent semantic drift

Why 1:1 matters

- Without a 1:1 analogue of the *same Admiral call*, independent SAS can quietly implement a **different rule**
- Examples: CTCv5 vs v4, a different EG source, a different treatment-emergent flag
- The risk is **silent semantic drift** — values look plausible, the derivation is not the same

Standards also move

- BRTHDTC / BIRTHDT dropped from many DM extracts (privacy)
- Newer OCCDS puts TRTEMFL on ADCM; PVA does not
- CTCAE v4 vs v5 changes grade boundaries
- A disagreement vs PVA is not automatically a SAS defect

Golden subjects vs Murphy^{*}

- CDISC-pilot / PVA is a small case set. A thin golden-subject panel can miss the rest.
- Remaining surprises here were Murphy-class: ADEG two EG files, HGB unit extract, CTCv4 vs v5 Hypokalemia / Hyponatremia.
- Edge-case checks and golden-subject testing were the QC this exploration *could* run — they are not a substitute for a pre-specified spec compare.
- AI uniquely enables **hundreds** of frozen rule cases as a counter-weight. If something can go wrong, it will.

^{*} Murphy's Law: If anything can go wrong, it will go wrong, at the worst possible time and in the worst possible manner.

Building a comprehensive golden-subject set

Cover the *rule space* — not every possible patient. That is how you avoid infinite combinations and still raise programming QC quality.

Standard CS methods

- **Partition** factors (PARAMCD, unit vs UNIT_CHECK, visit type, DTYPE, missing dates, TE window)
- **Boundaries** on cutoffs (HGB mmol/L vs g/L, K just below LLN, Na 130)
- **Covering arrays** (pairwise / selected 3-way) → hundreds of cases, not millions
- **Metamorphic** relations when there is no oracle; **mutation** to prove the suite can fail

AI's job vs the oracle

- AI drafts the factor table and fills the covering array from Admiral docs and past Murphy cases
- Do *not* invent 500 “realistic” patients with no expected ADaM
- Freeze each case: SDTM in, expected keys/values, one-sentence rule
- Industry effect: a counter-weight to Murphy — if it can go wrong, a named subject should catch it

Pre-planned vs as-programmed spex

A compare is only possible if both sheets use the **same data elements and the same degree of abstraction**.

Shared schema

- Large-pharma derived-data workbooks already look like this: derivation, inputs, by-vars, method, cutoff/unit, output var
- As-programmed / Layer 3 must emit *that* grain — not raw SAS and not a different template dialect
- Then pre-planned (SAP + HA + programming spex) can be row-matched to as-executed

When there is no published gold standard

- Trusted SAS macros (many studies, still running) or Admiral templates are a **first oracle**
- First industry step: new code vs that library, then vs golden subjects
- This exploration: spec-vs-spec was planned but curtailed (no Admiral spex found). The schema is what would make it executable

No gold standard in industry · R + SAS double programming

Industry default

- There is usually **no published gold standard** like PVA
- Two independent programs; discuss diffs
- First step is often compare new code vs a trusted prior set of macros or R functions

PVA as gold standard + cross-language QC

- PVA as gold standard made 1:1 analogues *easier* than industry — that gold standard is still not issue-free
- This exploration is **R + SAS double programming**: Admiral / PVA vs 1:1 SAS macros
- Large pharma already on R might adopt that pattern — plus 1:1 analogues to assist QC

Deliverables

What the proof-of-concept puts in the repo

- SAS ADaM builders + reusable macros that 1:1-parallel Admiral derivation steps
- Drivers for create / QC / golden subjects / edge tests (academic ODA where used)
- Optional R gold-standard export for dual-language compare (pharmaverse as oracle)
- As-programmed traceability: CSV and Excel worksheet `pva_as_programmed_spex.xlsx`

How to run (sketch)

1. Convert SDTM XPT → sdtm/*.sas7bdat (libname raw)
2. %include the ADaM builder driver
3. %include the metadata / Layer 3 traceability driver
4. %include the QC compare suite
5. (local R) optional transport / gold-standard export check

raw

SDTM input

adam

ADaM outputs

ref

R gold standard for compare

SAS OnDemand for Academics is a convenient student/academic runtime — not the name of the work.

Roadmap & call to action

First step completed

- Safety SAS macros reverse-engineered from R {admiral}, 1:1 where it mattered
- QC compare suite + as-programmed metadata / Layer 3 traceability

Next expansion

- Covering-array golden subjects (hundreds of frozen rule cases) + shared spex schema for pre-planned vs as-executed
- Complex efficacy endpoints & oncology domains
- Open-source the library on GitHub for industry collaboration

Ultimate goal: Give biotechs, sponsors, and CROs a reliable, open SAS alternative that combines modern pharmaverse design with the long-term stability of SAS.